

Module 2

Introduction to Hadoop: Introducing hadoop, Why hadoop, Why not RDBMS, RDBMS Vs Hadoop, History of Hadoop, Hadoop overview, Use case of Hadoop, HDFS (Hadoop Distributed File System), Processing data with Hadoop, Managing resources and applications with Hadoop YARN (Yet Another Resource Negotiator).

Introduction to Map Reduce Programming: Introduction, Mapper, Reducer, Combiner, Partitioner, Searching, Sorting, Compression.

TB1: Ch 5: 5.1-,5.8, 5.10-5.12, Ch 8: 8.1 - 8.8

Chapter -1

Introduction to Hadoop

Introducing Hadoop

- Big Data is a trending topic as **enterprises recognize the vast amount of untapped structured, semi-structured, and unstructured data available across networks.**

Every day:

- NYSE processes 1.5 billion trades.
- Facebook records 2.7 billion interactions.
- Google handles 24 petabytes of data.

Every minute:

- Facebook: 2.5 million content shares.
- Twitter: 300,000 tweets.
- Instagram: 220,000 photos uploaded.
- YouTube: 72 hours of video uploaded.
- Apple: 50,000 app downloads.
- Emails: 200 million sent.
- Amazon: \$80,000 in sales.
- Google: 4 million searches.

Every second:

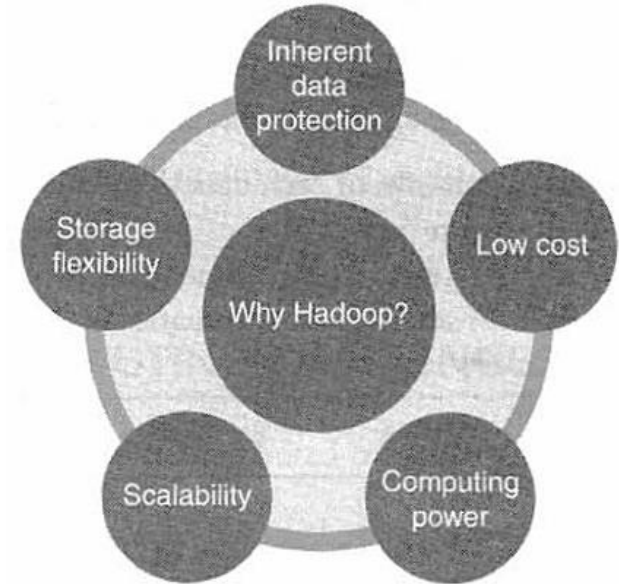
- Banks process 10,000+ card transactions.

Why Hadoop?

Hadoop is a popular technology because it can **quickly handle large and diverse data**.

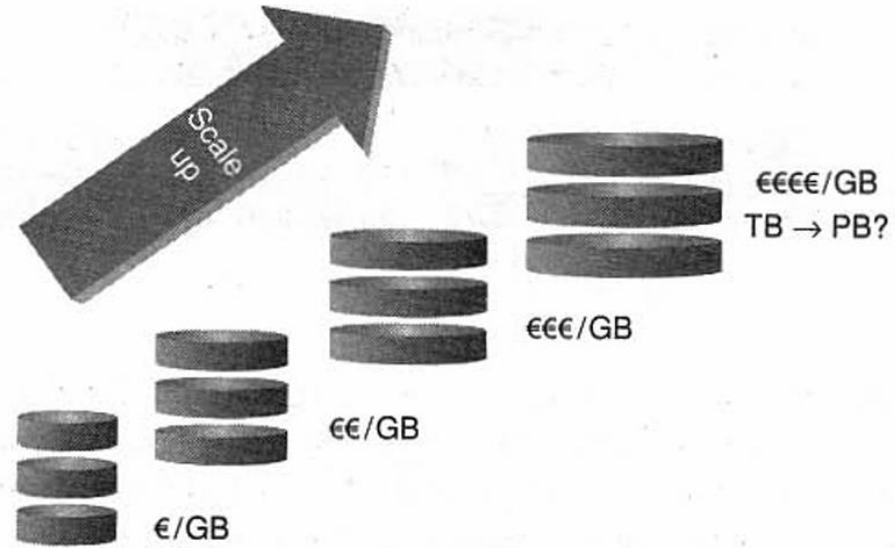
Its key advantages include:

- **Low Cost** – Open-source and uses inexpensive hardware.
- **High Computing Power** – Uses distributed computing to process large data efficiently.
- **Scalability** – Easily expands by adding more nodes.
- **Storage Flexibility** – Stores any type of data without pre-processing.
- **Data Protection** – Automatically manages failures and stores multiple copies of data.



Why not RDBMS?

- RDBMS is **not ideal** for **handling large files, images, and videos** because it is **designed for structured data**.
- It also struggles with **advanced analytics**, like **machine learning**.
- **As data grows**, using RDBMS becomes **expensive** due to **high storage and processing costs**.



RDBMS vs Hadoop

Parameter	RDBMS	Hadoop
System	Structured, table-based system	Node-based flat structure
Data Handling	Suitable for structured data	Supports structured and unstructured data (XML, JSON, text files, etc.)
Processing	Used for OLTP (Online Transaction Processing)	Used for big data and analytical processing
Choice	Best when data requires consistent relationships. For example, in a banking system, customer data, account details, and transactions must be linked correctly to maintain accuracy.	Best for big data processing without strict relationships. For example, analyzing large sets of social media posts, images, or sensor data, where relationships between data points are not strictly required.
Hardware Requirement	Requires expensive, high-end hardware for large data storage	Works with low-cost hardware (simple processor, network card, and hard drives)
Cost	\$10,000 to \$14,000 per terabyte	\$4,000 per terabyte

Distributed Computing Challenges

Prepared by: Darshan A,
Assistant Professor, Dept
of AI&DS, SJCT,
Chikkaballapur

- In a distributed system, multiple servers (computers) work together over a network. Since many hardware components are involved, there is always a chance that some may fail.

Why does hardware failure happen?

- Every computer has a hard disk to store data. Normally, a single hard disk may fail once in 3 years. Now, imagine a system with **1000 hard disks**—it is likely that a few of them will fail **every day**. This can lead to data loss if there is no backup.

How does Hadoop solve this?

- Hadoop prevents data loss using **Replication Factor (RF)**.

What is Replication Factor?

Replication Factor means that **multiple copies** of the same data are stored in different locations (servers) within the system. If one hard disk fails, the same data can still be accessed from another copy stored elsewhere.

For example:

- If **RF = 3**, it means each data block is stored in **three different servers**.
- If one server fails, Hadoop can use the other two copies to continue operations without losing data.

How to process this gigantic store of data?

- In a distributed system, **data is stored across multiple machines instead of a single one**. This creates a **challenge—before processing the data, we need to bring it together from different machines**.
- Hadoop solves this problem using **MapReduce Programming**. **Instead of moving all the data to one place for processing** (which would be slow and inefficient), MapReduce works in two steps:
- **Map Phase:** The data is divided into smaller chunks, and each chunk is processed separately on different machines.
- **Reduce Phase:** The processed results from all machines are combined to get the final output.
- This way, Hadoop ensures that large amounts of data are processed efficiently without overloading a single machine.

History of Hadoop

- Hadoop was created by **Doug Cutting**, who also developed **Apache Lucene**, a widely used text search library. It originated as part of **Apache Nutch**, an open-source web search engine project at Yahoo, and later became a part of the **Apache Lucene project**.

Why is it Called "Hadoop"?

- The name "**Hadoop**" is not an acronym or abbreviation. Doug Cutting named it after his child's **stuffed yellow elephant**. He chose this name because it was:
 - **Short and easy to spell.**
 - **Simple to pronounce.**
 - **Unique and unrelated to any existing technology.**
- This naming style continued in Hadoop's subprojects, which often have names inspired by animals. For example, "**Pig**" is another component of Hadoop.

Hadoop Overview

- ❑ Hadoop is an **open-source software framework** that helps in **storing and processing** huge amounts of data across multiple computers.
- ❑ It works on a **cluster of machines** (group of connected computers) using **commodity hardware** (affordable, easily available computers).

What does Hadoop do?

Hadoop mainly does two things:

- **Massive Data Storage** – Instead of storing data on a single computer, Hadoop splits the data into smaller parts and distributes them across many computers in the cluster. This allows it to store **huge amounts of data efficiently**.
- **Faster Data Processing** – Hadoop processes data in **parallel** using multiple machines at the same time. This means instead of one computer handling all the work, many computers work together, making processing **much faster**.

Key Aspects of Hadoop

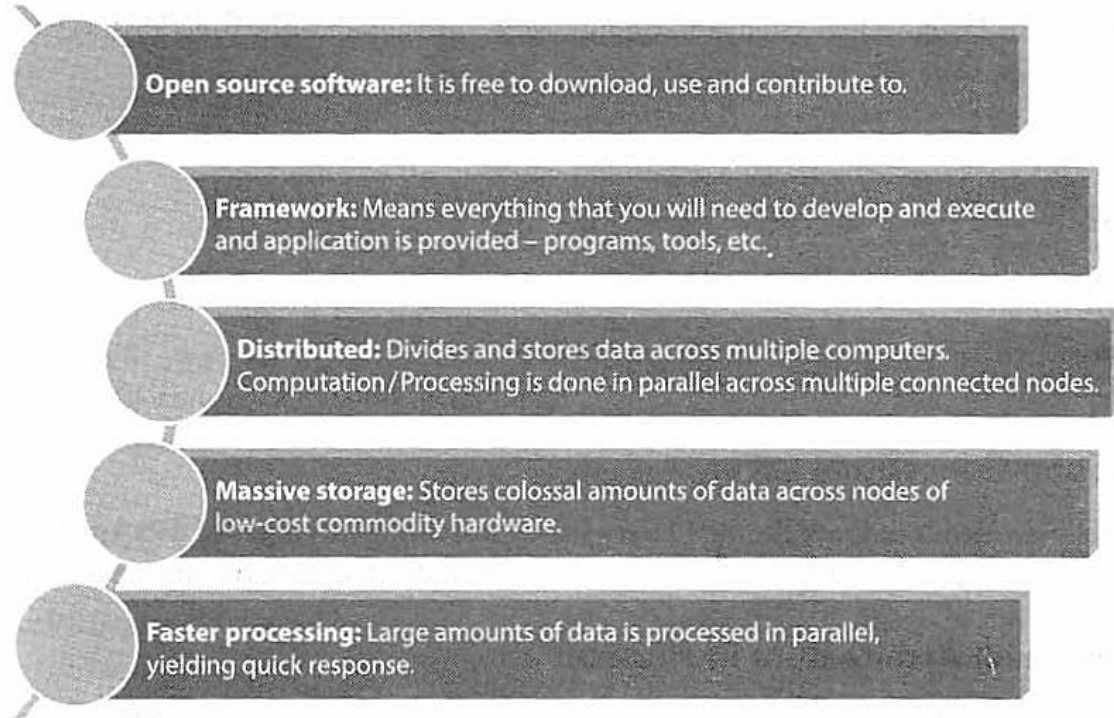


Figure 5.7 Key aspects of Hadoop.

Hadoop Components

- Hadoop has two main components that handle storage and processing:

- HDFS (Hadoop Distributed File System)**

- Storage Component:** Stores large amounts of data across multiple machines.
- Distributes Data:** Splits and distributes data across different nodes (computers).
- Redundant Storage:** Keeps multiple copies of data to prevent loss in case of failure.

- MapReduce**

- Computational Framework:** Helps in processing large datasets.
- Task Distribution:** Breaks down tasks into smaller parts and assigns them to different nodes.
- Parallel Processing:** Performs computations simultaneously to improve speed.

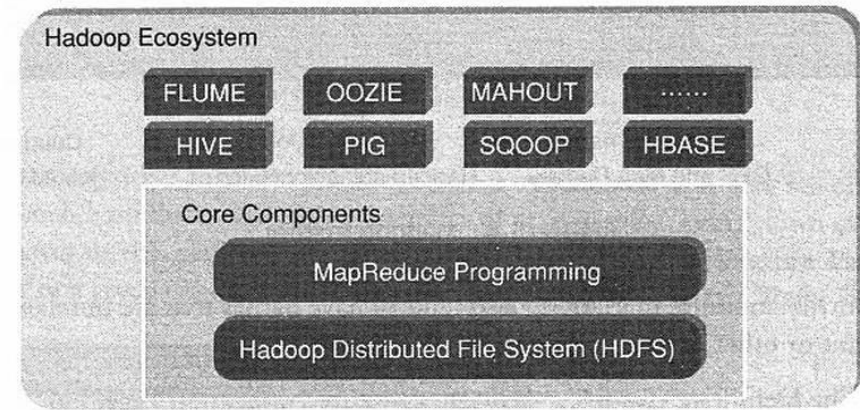


Figure 5.8 Hadoop components.

The **Hadoop Ecosystem** includes additional tools that enhance Hadoop's functionality. These tools help in **data processing, storage, and management**.

- **HIVE** – A SQL-like tool that helps users query and analyze large datasets stored in Hadoop.
- **PIG** – A scripting language that simplifies data processing in Hadoop.
- **SQOOP** – Transfers data between Hadoop and relational databases (like MySQL, PostgreSQL).
- **HBASE** – A NoSQL database that provides real-time access to Hadoop-stored data.
- **FLUME** – Collects and moves large amounts of streaming data (e.g., logs, social media feeds) into Hadoop.
- **OOZIE** – Manages and schedules Hadoop jobs and workflows.
- **MAHOUT** – Provides machine learning capabilities within Hadoop for tasks like clustering and recommendation.

These components work together to **improve the performance, usability, and efficiency of Hadoop** in handling big data.

Hadoop Conceptual Layers

Hadoop is divided into two main layers:

- **Data Storage Layer** – This layer is responsible for storing large amounts of data. It ensures that data is stored across multiple machines in a distributed manner.
- **Data Processing Layer** – This layer processes the stored data in parallel, meaning multiple parts of the data are processed simultaneously to generate useful insights quickly.

Hadoop High-Level Architecture

Hadoop follows a **Master-Slave Architecture**, meaning there is one central system (Master) that controls multiple worker systems (Slaves).

- **Master Node (NameNode)** – The central node that manages the storage and keeps track of where data is stored in the cluster.
- **Slave Nodes (DataNodes)** – These nodes store actual data and handle processing tasks.

In this setup, the **NameNode (Master)** directs **DataNodes (Slaves)** on where to store data and how to process it. This ensures efficient storage, quick data processing, and fault tolerance (if one DataNode fails, another can take over).

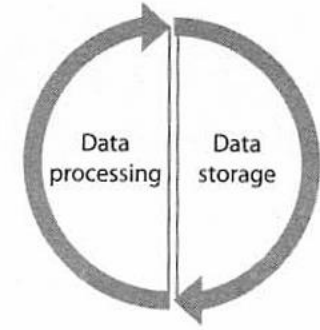


Figure 5.9 Hadoop conceptual layer.

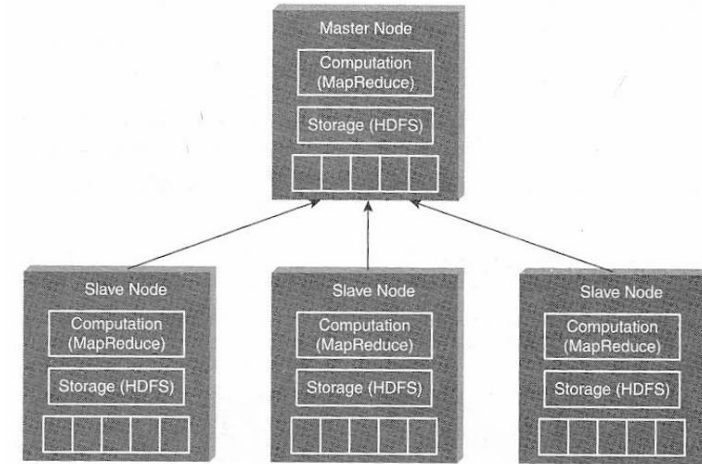


Figure 5.10 Hadoop high-level architecture.

Key Components of the Master Node in Hadoop

- The **Master Node** in Hadoop is responsible for managing both data storage and data processing. It has two main components:

1. Master HDFS (Hadoop Distributed File System)

- Manages how data is stored in Hadoop.
- **Working :**
 - It divides (partitions) large data files into smaller parts.
 - These smaller parts are then stored across multiple **Slave Nodes** (also called **DataNodes**).
 - It keeps a record of where each part of the data is stored.

2. Master MapReduce

- Manages how data is processed in Hadoop.
- **Working:**
 - It assigns data processing tasks to **Slave Nodes** based on where the data is stored.
 - It ensures the tasks run efficiently by distributing the workload.
 - It schedules the tasks and monitors their progress.

Use case of Hadoop

ClickStream Data

- ClickStream data (mouse clicks) helps businesses understand customer purchasing behavior.
- **ClickStream analysis** using Hadoop offers three key benefits:
 - ❖ **Data Integration** – Hadoop combines ClickStream data with other sources like customer demographics, sales, and advertising data to provide deeper insights.
 - ❖ **Scalability** – Hadoop stores years of data at a low cost, enabling long-term trend analysis.
 - ❖ **Analytics Support** – Tools like Apache Pig and Hive help organize and refine ClickStream data for better visualization and decision-making.
- This helps businesses optimize websites, promotions, and marketing strategies effectively.

HDFS (Hadoop Distributed File System)

Some key points of HDFS are as follows:

- **Storage Component:** HDFS is the main storage system used in Hadoop to **store large amounts of data**.
- **Distributed System:** It **splits large files into smaller parts** (blocks) and **distributes** them across multiple computers (nodes).
- **Inspired by Google File System:** HDFS is **designed** based on **Google's approach to handling large-scale data storage**.
- **High Performance:** It is optimized for handling big data efficiently by **storing large blocks of data** and **processing it close to where it is stored**.
- **Replication for Fault Tolerance:** Each file is **stored in multiple copies** across **different nodes**, making it resistant to software or hardware failures.
- **Automatic Recovery:** If a **node fails**, **HDFS automatically replicates** and **restores the lost data** from other copies.
- **Best for Large Files:** HDFS is ideal for **reading and writing large files** (in gigabytes or more).

- **Works on Existing File Systems:** **HDFS** runs on **top of standard file systems** like **ext3 (Third Extended Filesystem)** and **ext4 (Fourth Extended Filesystem)** (used in Linux).

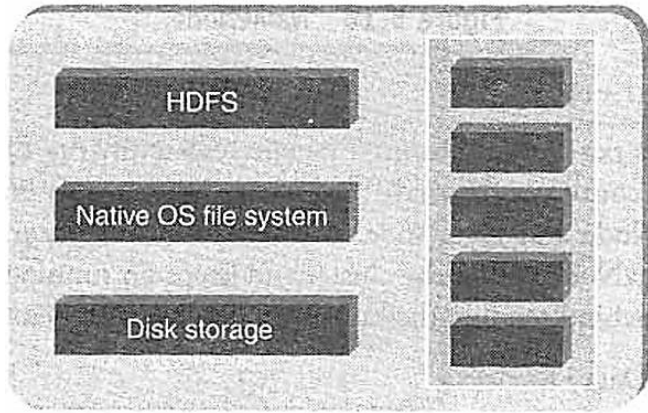


Figure 5.13 Hadoop Distributed File System.

- When data is written to HDFS, it is broken into blocks and stored across multiple nodes in a distributed environment.
- These **blocks are stored as files** in the **underlying Linux file system** (ext3, ext4, or others).
- The **Linux file system** handles the actual **disk-level operations** like reading, writing, and organizing data.

Windows file systems are **NTFS(New Technology File System)**, **FAT32 (File Allocation Table 32-bit)**, **exFAT (Extended File Allocation Table)**, **ReFS (Resilient File System)**

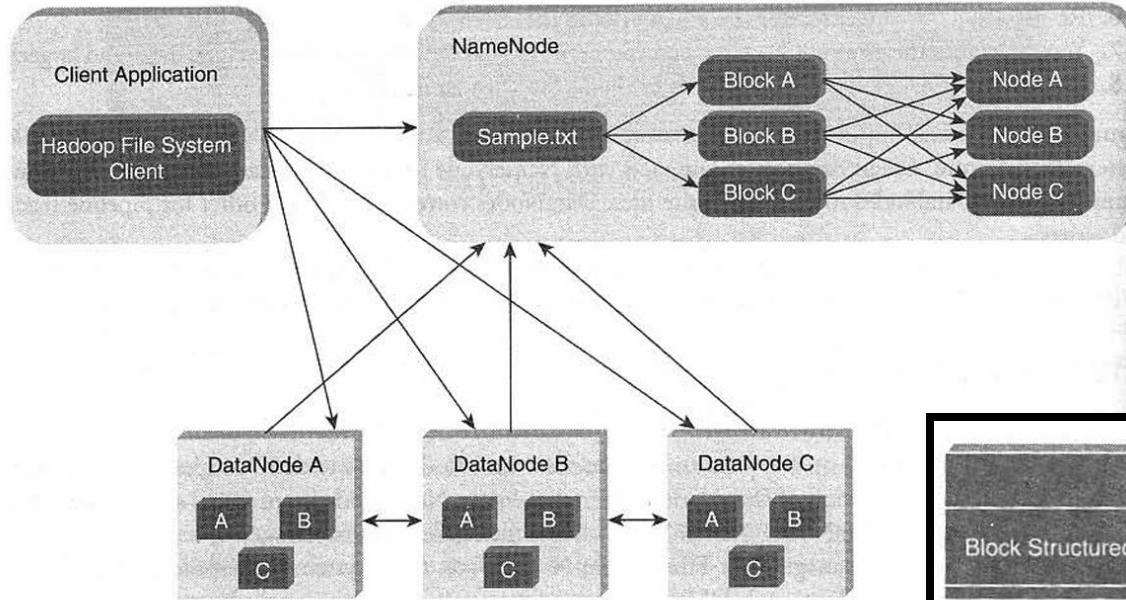


Figure 5.15 Hadoop Distributed File System Architecture.

Figure 5.15 is a HDFS architecture. **Client application** interacts with **namenode** for **metadata related activities** and **communicates with datanode to read and write files**. **Datanodes converse with each other for pipeline reads and write**.

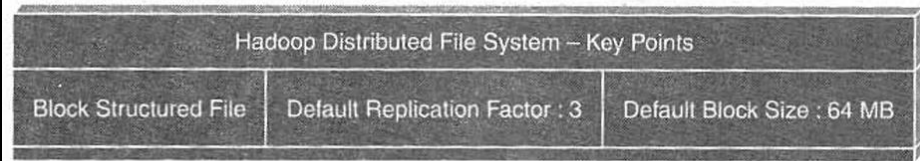


Figure 5.14 Hadoop Distributed File System – key points.

Example: If a file named "**Sample.txt**" is **192 MB** in size, HDFS will:

1. Split it into **three blocks** of **64 MB** each (since the **default block size is 64 MB**).
2. Distribute these blocks across different nodes.
3. Replicate them based on the **default replication factor (typically 3)**, ensuring data reliability.

HDFS Daemons

NameNode in HDFS

1. **Manages File System:** NameNode is the **master** of HDFS and keeps track of all files and directories.
2. **Breaks Large Files into Blocks:** HDFS splits large files into **blocks** and distributes them across multiple **DataNodes**.
3. **Tracks DataNodes Using Racks:**
 - A **rack** is a group of DataNodes within a cluster.
 - NameNode assigns **rack IDs** to DataNodes and tracks where each block is stored.
4. **File System Namespace:**
 - The **namespace** includes a list of all files, directories, and block locations in HDFS.
 - NameNode stores this information in a special file called **FsImage**.
5. **Transaction Log (EditLog):**
 - Every file operation (**create, delete, modify**) is recorded in **EditLog** (a transaction log).
 - When NameNode restarts, it reads **FsImage** and applies all transactions from **EditLog** to update the file system state.
 - After updates, it saves the new **FsImage** and clears the old **EditLog**.
6. **Single NameNode Per Cluster:**
 - There is only **one NameNode** managing the entire HDFS cluster.
 - It does not store actual file data—only metadata (file locations and properties).

- If you store a file in HDFS:
 - NameNode **breaks it into blocks** and assigns them to DataNodes.
 - It **records file metadata** (e.g., block locations, size, and permissions) in **FsImage**.
 - If a new file is added, the change is **logged in EditLog** for future updates.

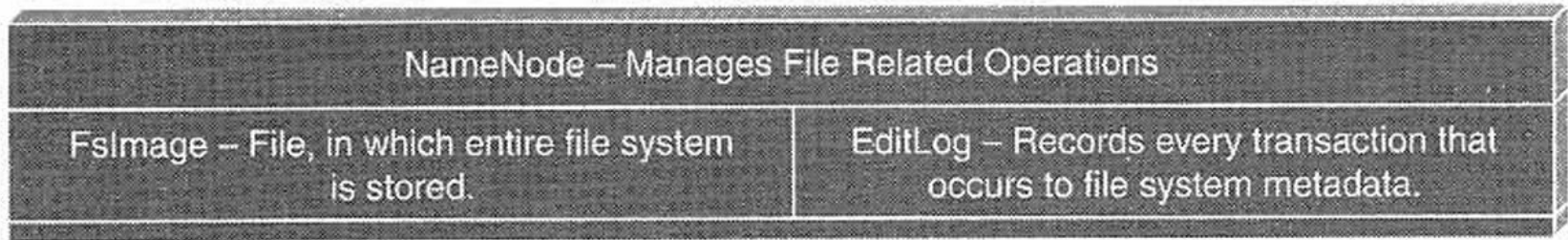


Figure 5.16 NameNode.

HDFS Daemons

Datanode in HDFS

- A Hadoop cluster has **many DataNodes** to store and manage file blocks.
- During **file read and write**, DataNodes **exchange data** to ensure efficient processing.
- Each DataNode sends a **heartbeat signal** to the **NameNode** at regular intervals. This confirms that the DataNode is **active and functioning**.
- If a DataNode **stops sending heartbeats**, the NameNode assumes it has **failed**. The **NameNode replicates the lost data from other copies** to maintain **fault tolerance**.

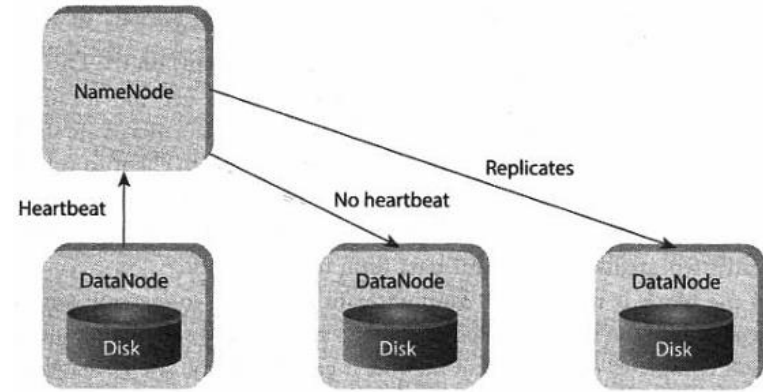


Figure 5.17 NameNode and DataNode Communication.

HDFS Daemons

Secondarynode in HDFS

- The secondary namenode **takes a snapshot of HDFS metadata at intervals specified** in the hadoop configuration.
- Since the **memory requirements of secondary namenode** are the **same as namenode**, it is better to **run namenode and secondary namenode on different machines**.
- In **case of failure** of the namenode, the **secondary namenode** can be **configured manually to bring up the cluster**.
- However the secondary namenode **does not record any real-time changes** that happen to the HDFS metadata.

Anatomy of file read

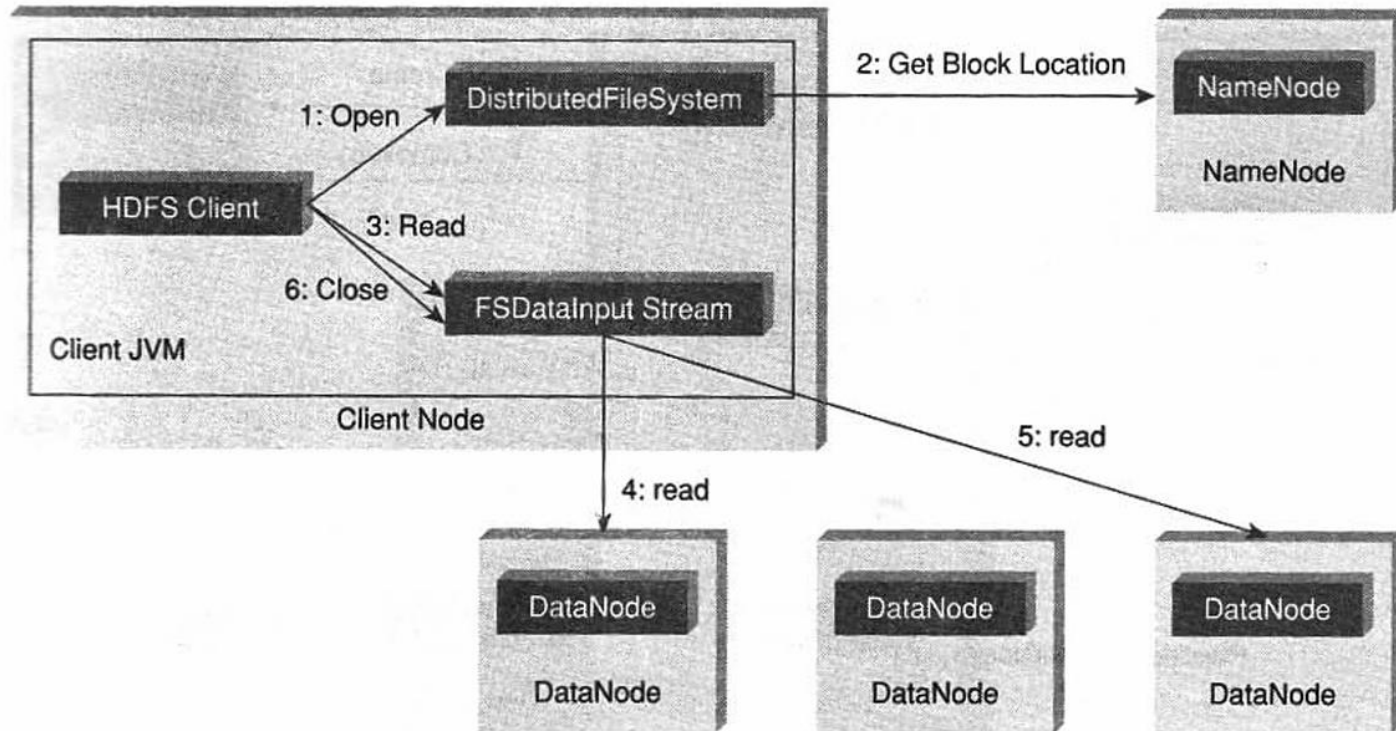


Figure 5.18 File Read.

1. The client calls **open()** on **DistributedFileSystem** to read a file.
2. The **DistributedFileSystem** contacts the **NameNode** to find out **where the file's data blocks are stored**. The **NameNode** responds with the addresses of **DataNodes** storing the required blocks. The **client receives a stream (FSDataInputStream – It is responsible to read data in chunks)** to start reading the file.
3. The client calls **read()** on the **DFSInputStream**, which **contains the addresses of the DataNodes**. It **chooses the closest DataNode** for the first block and starts reading.
4. The client **continuously calls read()** to fetch data from the DataNode.
5. When the **end of a block** is reached, the **connection with the DataNode is closed**. The client **finds the next best DataNode** and continues reading the next block. This process **repeats** for all remaining blocks.
6. After reading the full file, the client **calls close()** on **FSDataInputStream** to close the connection.

- **FSDataInputStream** is a special **input stream** in HDFS that allows the client to **read a file block by block**.
- It helps the client **retrieve data efficiently** from different **DataNodes** without needing to manage the process manually.

Anatomy of file write

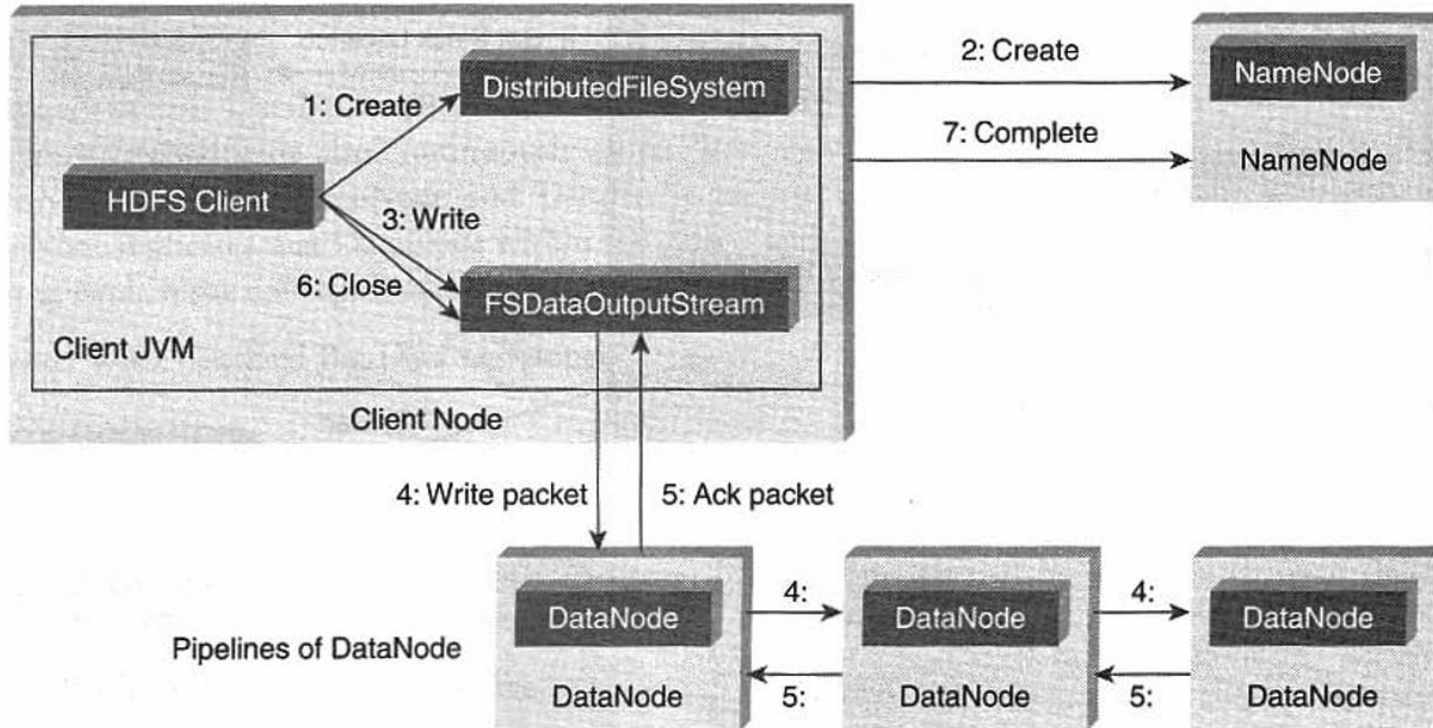


Figure 5.19 File Write.

1. The client calls **create()** on **DistributedFileSystem** to create a new file.
2. The **DistributedFileSystem** makes an **RPC call** (Remote Procedure Call) to the **NameNode**. **NameNode checks** whether the file already exists. If valid, it **creates a file entry** but **does not yet assign data blocks**. The system returns **FSDDataOutputStream** to the client **for writing data**.
3. The **client writes data** to the **DFSOutputStream**, which **splits the data into packets**. These packets go into a queue called **data queue**. A process called **DataStreamer** **picks up packets** and **requests the NameNode to assign DataNodes for storage**. DataNodes are selected based on **replication factor** (default = 3). The selected **DataNodes form a pipeline** to receive and replicate the data.
4. **DataStreamer sends packets** to the **first DataNode** in the pipeline. The **first DataNode stores** the packet and **forwards it to the second DataNode**. The **second DataNode stores** the packet and **forwards it to the third DataNode**. This process ensures **replication** across multiple nodes.

5. Along with the **data queue**, DFSOutputStream maintains an **ack queue** (acknowledgment queue). A packet **remains in the ack queue** until **all DataNodes in the pipeline confirm** they have successfully stored it.
6. When the client finishes writing, it **calls close()** on the stream. This **flushes all remaining packets** to the DataNode pipeline. The system **waits for acknowledgments** from all DataNodes. After confirmation, the client **informs the NameNode** that the file creation is complete.

Hadoop default replica placement strategy

- Hadoop follows a **smart strategy** to store copies (replicas) of data across different nodes and racks to ensure **fault tolerance and reliability**.
- When a file is saved in HDFS, **three copies (replicas) of each block** are stored as follows:
 1. **First replica** → **Stored on the same node as the client** (if the client is inside the cluster).
 2. **Second replica** → **Placed on a different rack** (a rack is a group of nodes).
 3. **Third replica** → **Stored on the same rack as the second replica, but on a different node.**
- Advantages of this strategy:
 1. **High Availability:** If one node fails, other copies exist on different nodes/racks.
 2. **Better Fault Tolerance:** Since at least one replica is on a different rack, data is safe even if a whole rack fails.
 3. **Efficient Network Usage:** Keeps one local copy for fast access while balancing network load across racks.
- Once the replica locations are decided, **a pipeline is built** to send the data to these nodes in order. The first node stores the data and forwards it to the second, which then sends it to the third.

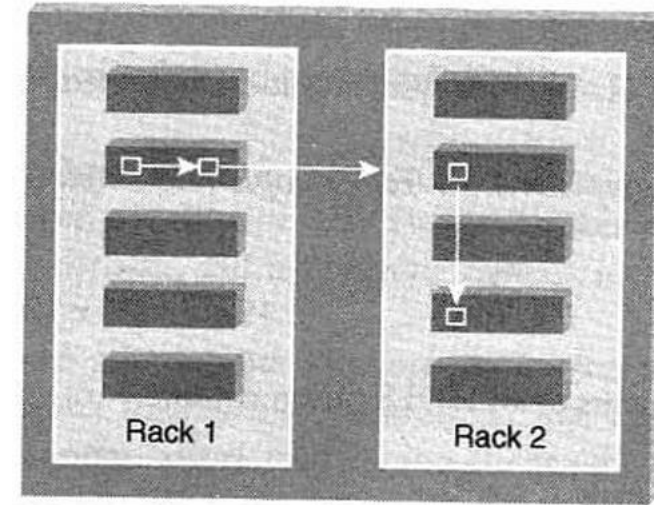


Figure 5.20 Replica Placement Strategy.

Working with HDFS commands

Objective: To get the list of directories and files at the root of HDFS.

Act:

hadoop fs -ls /

`hadoop fs` → Interacts with the Hadoop file system (HDFS).

`-ls` → Lists the files and directories (similar to `ls` in Linux).

`/` → Refers to the root directory in HDFS.

Objective: To get the list of complete directories and files of HDFS.

Act:

hadoop fs -ls -R /

`hadoop fs` → Interacts with the Hadoop file system (HDFS).

`-ls` → Lists files and directories.

`-R` → Recursive mode, which means it lists not only the files in the root (`/`) but also inside all subdirectories.

`/` → Specifies the root directory in HDFS.

Objective: To create a directory (say, sample) in HDFS.

Act:

hadoop fs -mkdir /sample

Objective: To copy a file from local file system to HDFS.

Act:

hadoop fs -put /root/sample/test.txt /sample/test.txt

Objective: To copy a file from HDFS to local file system.

Act:

hadoop fs -get /sample/test.txt /root/sample/testsample.txt

Objective: To copy a file from local file system to HDFS via copyFromLocal command.

Act:

hadoop fs -copyFromLocal /root/sample/test.txt /sample/testsample.txt

hadoop fs → Interacts with the Hadoop file system (HDFS).

-copyFromLocal → Copies a file from the local file system to HDFS.

/root/sample/test.txt → This is the **source file** (located in the local file system).

/sample/testsample.txt → This is the **destination file path** in HDFS.

Objective: To copy a file from Hadoop file system to local file system via copyToLocal command.

Act:

```
hadoop fs -copyToLocal /sample/test.txt /root/sample/testsample1.txt
```

Objective: To display the contents of an HDFS file on console.

Act:

```
hadoop fs -cat /sample/test.txt
```

Objective: To copy a file from one directory to another on HDFS.

Act:

```
hadoop fs -cp /sample/test.txt /sample1
```

Objective: To remove a directory from HDFS.

Act:

```
hadoop fs-rm-r /sample1
```

Special features of HDFS

- **Data Replication** – The client doesn't need to track file blocks. Instead, it is automatically directed to the nearest data replica for better performance.
- **Data Pipeline** – When writing data, the client sends the first block to a DataNode, which then forwards it to the next node in the pipeline. This continues until all blocks and their replicas are written to disk.

Processing Data with Hadoop

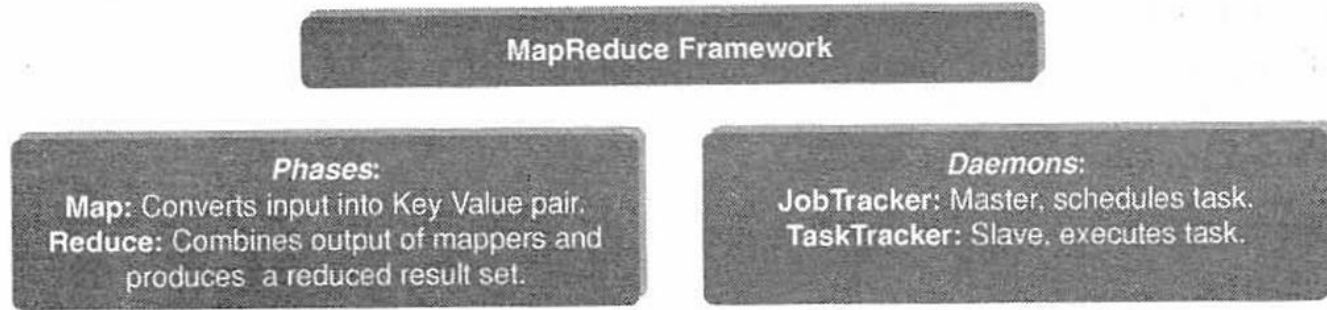


Figure 5.21 MapReduce Programming phases and daemons.

- **MapReduce** is a **software framework** for processing large datasets in parallel. It splits input data into independent chunks, processes them using **Map** tasks, and stores intermediate results locally. The framework automatically shuffles, sorts, and provides this output to **Reduce** tasks, which combine the results to generate the final output.
- The **Hadoop Distributed File System (HDFS)** and **MapReduce framework** run on the same nodes for data locality, improving throughput.
- Two key components manage the process:
 - JobTracker (Master):** Schedules, monitors, and re-executes failed tasks.
 - TaskTracker (Slave):** Executes the assigned tasks.
- **Job configuration** includes **MapReduce functions, input/output locations, and job parameters**. The **JobTracker** receives jobs from the **Hadoop job client**, **schedules tasks, monitors progress, and updates the client**.

MapReduce Daemons

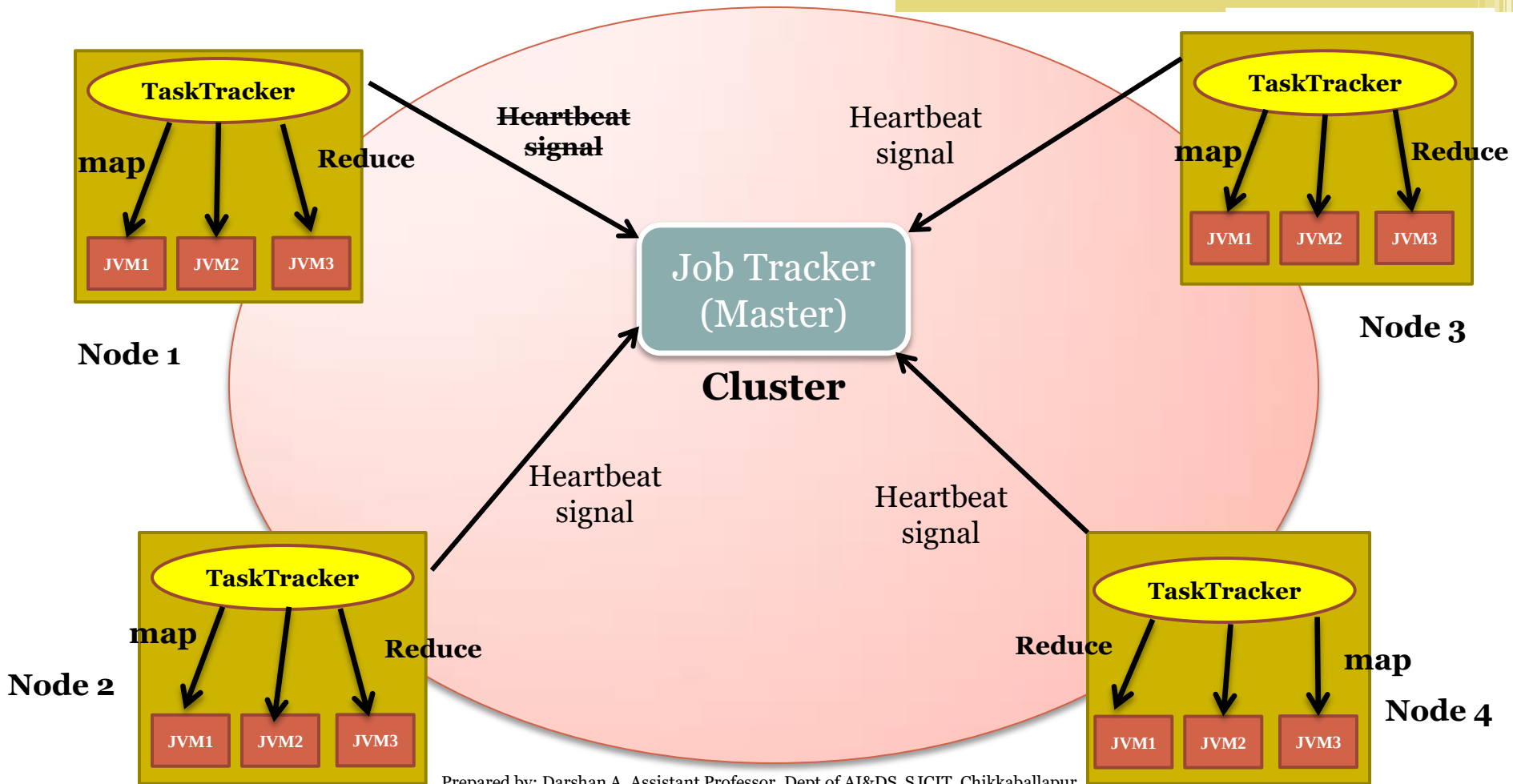
1) JobTracker (Master)

- It provides a **connectivity between hadoop and your application.**
- Acts as the **manager of all jobs** in a Hadoop cluster.
- When a mapreduce job is submitted, the JobTracker decides:
 - Which task should be assigned to which node in the cluster.
 - Monitors all running tasks.
 - If a task fails, it reassigns the task to another node.
- **Only one JobTracker** exists **per Hadoop cluster.**
- It **ensures the smooth execution** of the **entire MapReduce process.**

MapReduce Daemons

2) Task Tracker (Worker)

- This daemon is **responsible for executing individual tasks** that is assigned by the job tracker.
- Works as the **executor of tasks assigned** by the JobTracker.
- **Each worker node (slave)** in the **cluster** has **one TaskTracker**.
- The **TaskTracker** can **run multiple map and reduce tasks** in **separate JVMs** (Java Virtual Machines).
- It **regularly updates** the **JobTracker** by **sending** a **heartbeat signal**.
- **If the JobTracker does not receive** a **heartbeat**, it **assumes** the **TaskTracker** has **failed** and **reassigns** the **task to another node**.



How does mapreduce work?

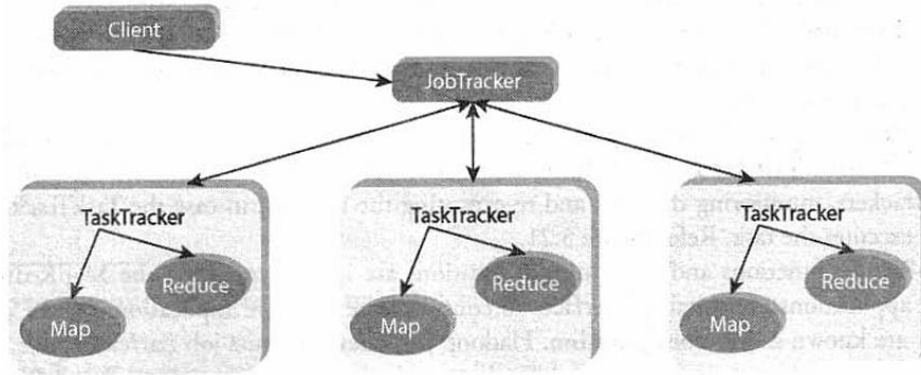


Figure 5.22 JobTracker and TaskTracker interaction.

- In the given block diagram, there are two mappers and one reducer. Each mapper works on the partial dataset that is stored on that node and the reducer combines the output from the mappers to produce the reduced result set.

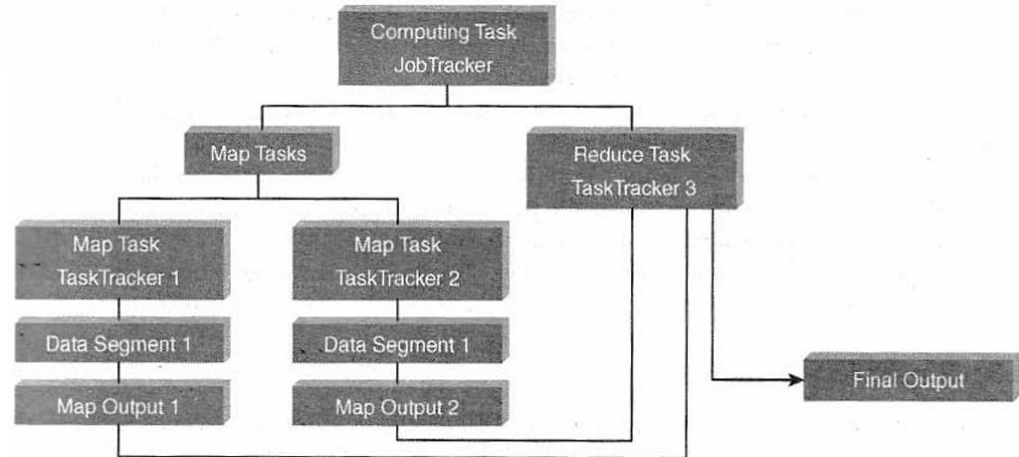


Figure 5.23 MapReduce programming workflow.

Working model of mapreduce programming

1. Splitting the Data:

- The input data is **broken into smaller chunks** so that multiple tasks can work on it at the same time.

2. Creating Master and Worker Processes:

- A **master process** is created to **control and assign tasks**.
- Multiple **worker processes** are started on different machines to handle the tasks.

3. Mapping Phase:

- Each **worker (map task)** reads its assigned **chunk of data**.
- The **map function** processes this data and converts it into **key-value pairs**.

4. Partitioning the Data:

- The map worker uses a **partitioner** to decide **which reducer** should receive the output.
- This ensures that data related to the same key is sent to the same reducer.

5. Shuffling and Sorting:

- After all map workers finish, the **reduce workers collect the key-value pairs**.
- The data is **shuffled (grouped by key) and sorted** for easier processing.

6. Reducing Phase:

- The **reduce function** processes each unique key and **aggregates results**.

7. Completion:

- Once all **reducers finish**, the **master gives control back** to the user program.
- The user can now access the final processed results.

Managing resources and applications with Hadoop YARN (Yet Another Resource Negotiator)

What is hadoop YARN?

- YARN stands for **Yet Another Resource Negotiator** and is a part of **Hadoop 2.x**.
- It is a **resource management framework** that allows Hadoop to handle **multiple types of applications**, not just MapReduce.
- Now, Hadoop can handle **Batch Processing, Interactive Queries, Online Processing, Streaming, and Graph Processing** efficiently.

Limitations of Hadoop 1.0

- Single NameNode Problem

- Only **one NameNode** managed the entire Hadoop cluster.
- If this **NameNode failed**, the **entire system** could **stop working** (**single point of failure**).

- Limited to Batch Processing

- It could only handle **batch jobs**, meaning it processed large amounts of data **in one go**.
- It was **slow for real-time tasks**.

Managing resources and applications with Hadoop YARN (Yet Another Resource Negotiator)

- **Not Good for Quick Queries (Interactive Analysis)**

- You couldn't ask questions and get **instant answers** from large datasets.

- **Not Suitable for Machine Learning & Graph Processing**

- Machine learning and graph-based tasks need **more memory and faster processing**. Hadoop 1.0 **struggled** with these workloads.

- **Poor Resource Management**

- The system had **fixed slots** for **map tasks** and **reduce tasks**.
- If **map slots were full** but reduce slots were **empty**, resources were wasted.
- This led to **inefficient resource utilization**.

HDFS Limitation in Hadoop 1.0

- The **NameNode** in Hadoop **stores file metadata** (information about files and directories) **in main memory (RAM)**.
- If there are **too many files**, the **NameNode's memory gets full** and cannot handle more files.
- This **slows down the system** and may cause failures.
- Even though **RAM is cheaper today**, it still has a **limit** on how much data it can store.
- If a **single NameNode** handles **everything**, it can become **overloaded** when the system grows.
- In Hadoop 2.x, this is resolved with the help of **HDFS Federation**.

Hadoop 2: HDFS

Major Components of HDFS 2:

- **Namespace Service** – Manages **file-related operations** like creating, modifying, and organizing files/directories.
- **Block Storage Service** – Manages **data storage**, handles **DataNodes**, and takes care of **replication** for fault tolerance.

Features of HDFS 2:

1) Horizontal Scalability:

- Uses **multiple NameNodes** to handle more data efficiently.
- Each NameNode works **independently** without needing coordination.
- All **DataNodes** are shared among NameNodes.

2) High Availability:

- Hadoop 1.x had **only one NameNode** to manage the system. If this **single NameNode failed**, the entire Hadoop cluster would **stop working**. This caused **downtime** and **data access issues**. **Solution in Hadoop 2.x – Active-Passive NameNode Setup**. To ensure **high availability**, Hadoop 2.x introduced the **Active-Passive NameNode architecture** with **automatic failover**.
- Two NameNodes Exist:
 - a) Active NameNode** → **Manages all operations** (reads, writes, metadata updates).
 - b) Passive (Standby) NameNode** → **Remains idle** but stays updated.
- All **namespace changes** (file modifications, deletions, etc.) are recorded in a **shared NFS storage** (Network File System). This ensures that both **Active and Passive NameNodes** have the same updated metadata.

- If the **Active NameNode** fails, the **Passive NameNode** **automatically** takes over. It **becomes the new Active NameNode** and starts handling operations. The transition happens **seamlessly**, ensuring that the system **continues running without downtime**.
- At any time, **only one NameNode writes to storage** to prevent conflicts. The **Passive NameNode only reads updates** until it becomes active.

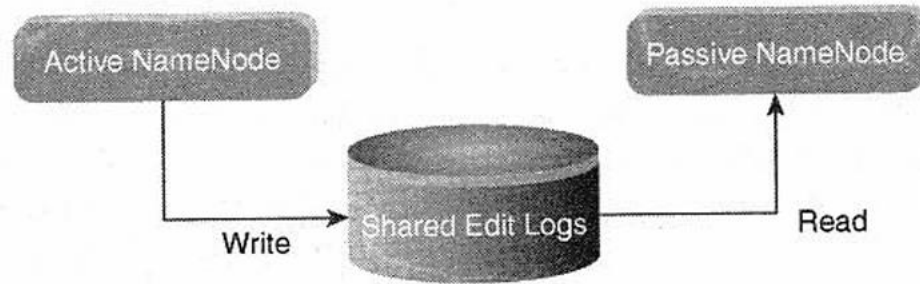


Figure 5.26 Active and Passive NameNode interaction.

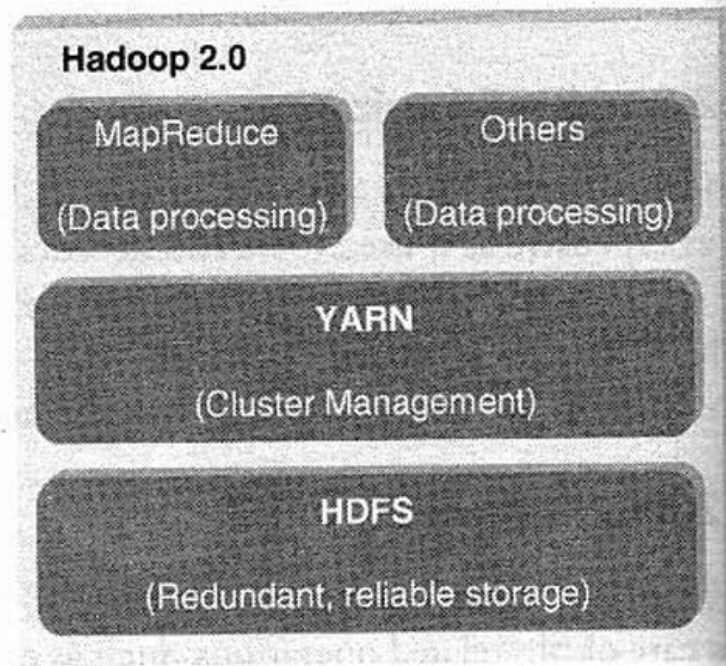
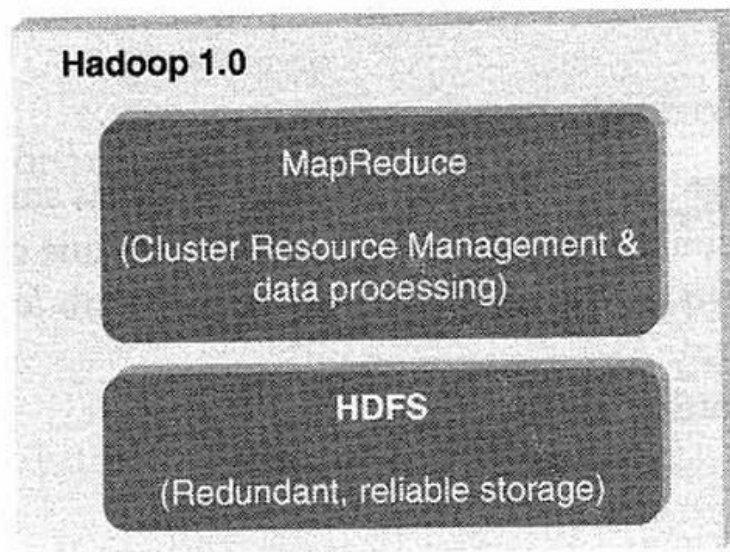


Figure 5.27 Hadoop 1.x versus Hadoop 2.x.

Hadoop 2 YARN: Taking Hadoop beyond Batch

- **YARN (Yet Another Resource Negotiator)** is a key feature in **Hadoop 2.x** that **manages resources** efficiently. It **allows multiple applications** to run on Hadoop, not just **batch processing like MapReduce**.
- **Hadoop 2.x** introduced **YARN**, allowing **better resource utilization** and **support for different types of processing**.
- **Stores All Data in One Place** → You don't need separate systems for different types of workloads.
- **Supports Multiple Processing Models** → **Hadoop** is **no longer limited to batch jobs**; it can handle **real-time, streaming, interactive, and graph-based processing**.
- It **efficiently allocates resources**, ensuring **smooth and optimized processing**.
- **Improves job scheduling and priority handling** for **better reliability and efficiency**.
- **Yahoo** originally designed **YARN** to **improve Hadoop's performance and flexibility**.

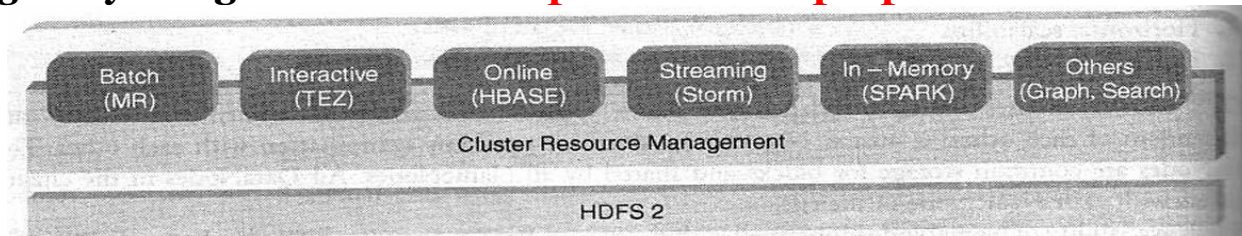


Figure 5.28 Hadoop YARN.

Fundamentals Idea

Daemons of YARN architecture are:

a) Global Resource Manager (Manages System-Wide Resources)

- **Allocates resources** among different applications.
- It has two main parts:

(a) **Scheduler** – **Decides how much CPU, memory, and other resources each application gets** but does not track their status.

(b) **Application Manager** –

- Accepts **job submissions**.
- **Allocates containers** (a unit of resources like CPU and memory) for applications.
- **Restarts failed Application Masters.**

b) NodeManager (Per-Machine Resource Monitor & Executor)

- Runs on each machine in the cluster.
- **Starts and monitors containers** (where applications run).
- **Tracks resource usage** (CPU, memory, disk, etc.) and reports to the **Resource Manager**.

c) ApplicationMaster (Manages a Specific Application)

- **Each application gets its own ApplicationMaster**.
- **Requests resources** from the **Resource Manager**.
- Works with **NodeManager** to **run and monitor tasks**.

Terminologies

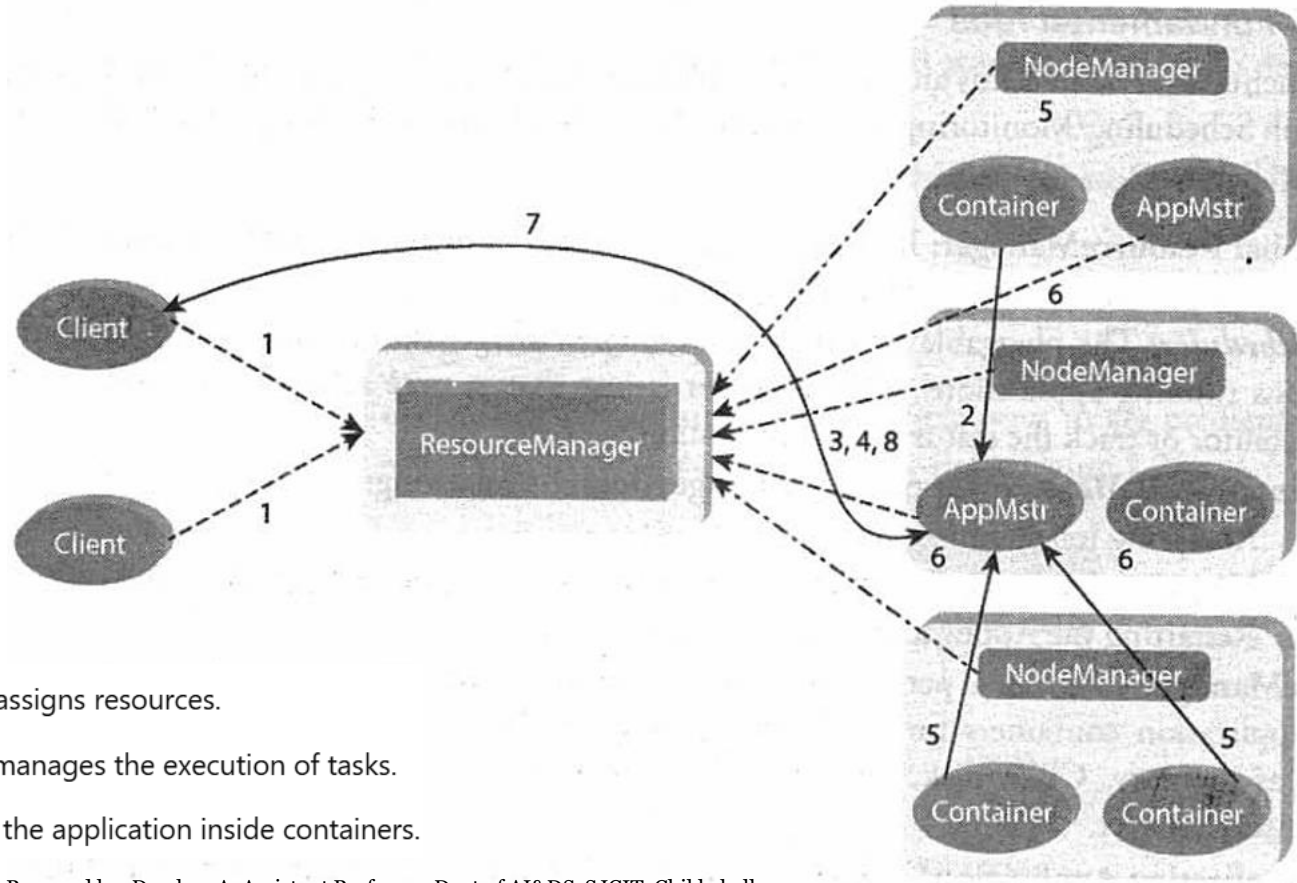
1. Application

- An **application** is a job that is submitted to the **Hadoop framework**.
- Example: A **MapReduce Job**, which processes large-scale data in a distributed manner.

2. Container

- A **container** is the basic unit of resource allocation in YARN.
- It provides **fine-grained resource allocation** based on multiple factors such as **memory, CPU, disk, and network**.
- Example allocations:
 - **Container 0**: 2GB Memory, 1 CPU
 - **Container 1**: 1GB Memory, 6 CPUs

YARN Architecture



The **Resource Manager** assigns resources.

The **ApplicationMaster** manages the execution of tasks.

The **NodeManager** runs the application inside containers.

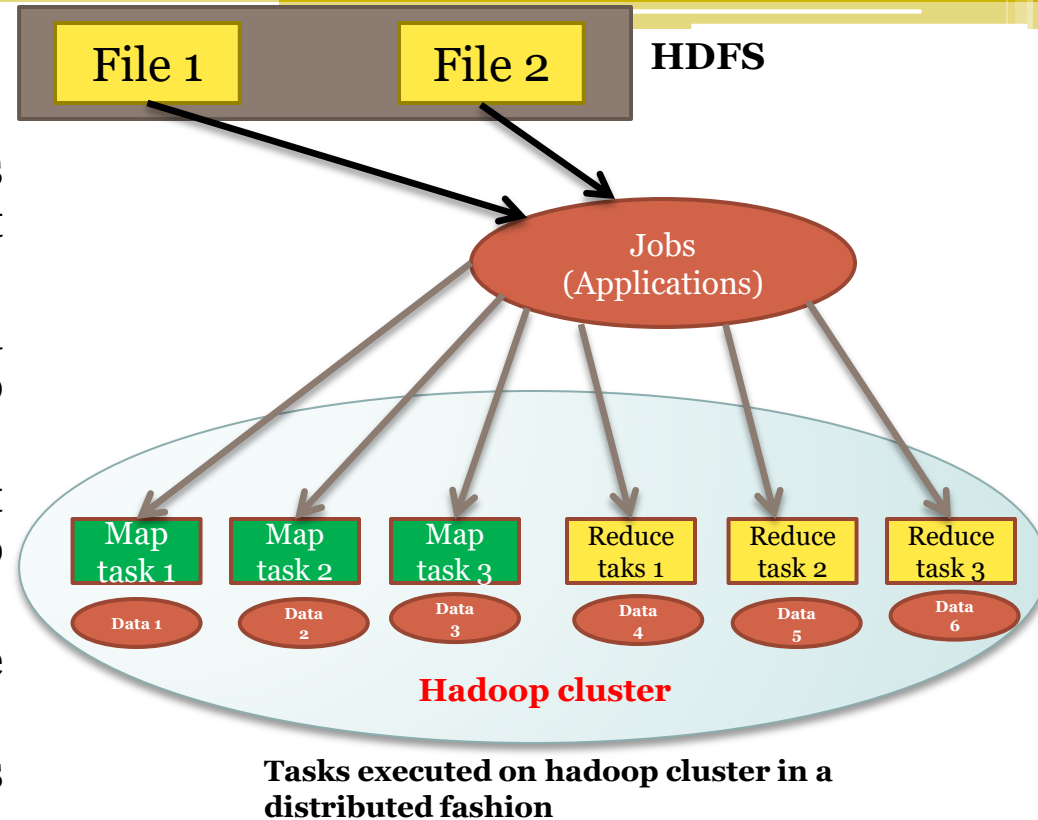
- 1) **Client submits the job** → The user submits an application with all the details needed to run it, including the instructions for launching the **ApplicationMaster**.
- 2) **Resource Manager assigns a container** → It selects a container (a unit of resources like CPU & memory) to launch the **ApplicationMaster**.
- 3) **ApplicationMaster registers with Resource Manager** → This helps the client check the job's status directly with the **Resource Manager**.
- 4) **ApplicationMaster requests more containers** → It negotiates with the **Resource Manager** to get the necessary resources (CPU, memory) for running the job.
- 5) **Containers are assigned and launched** → Once allocated, the **ApplicationMaster** sends instructions to **NodeManager**, which starts the **containers**.
- 6) **NodeManager runs the application** → It executes the code inside the container and sends **progress updates** to the **ApplicationMaster**.
- 7) **Client communicates with ApplicationMaster** → The user can check the status of their job by talking to the **ApplicationMaster** directly.
- 8) **ApplicationMaster completes and shuts down** → After finishing the job, the **ApplicationMaster** deregisters from the **Resource Manager**, freeing up resources.

Chapter 2

Introduction to mapreduce programming

Introduction

- In mapreduce programming, jobs (applications) are split into a set of map tasks and reduce tasks.
- These tasks are executed in a distributed fashion on hadoop cluster.
- Each task processes small subset of data that has been assigned to it.
- This way, hadoop distributes the load across the cluster.
- Mapreduce job takes a set of files that is stored in HDFS (Hadoop Distributed File System) as input.



- Map task takes care of **loading, parsing, transforming, and filtering.**

- ❑ **Loading:-** Map task loads data from HDFS. It reads the input files **line by line, row by row**, or in another format based on the file type.
- ❑ **Parsing:-** After loading, the **Map task parses** (breaks down) the data into smaller components. If the input is text, **parsing** means splitting the sentence into words
- ❑ **Transforming:-** Map task **transforms the parsed data into key-value pairs**
- ❑ **Filtering:-** Map task **filters out unwanted data. It removes unnecessary words, null values, or unwanted entries.**

- The responsibility of reduce task is **grouping and aggregating data** that is produced by map tasks to generate final output.
- ❑ **Grouping:-** Reduce task **groups all values** that have the same key.
- ❑ **Aggregating:-** The Reduce task aggregates (summarizes) the grouped values. Aggregation means performing operations like **sum, count, average, max, min, etc.**

- Each **map task** is broken down into the following phases:
 - a) **RecordReader**
 - b) **Mapper**
 - c) **Combiner**
 - d) **Partitioner**
- The **output** produced by **map task** is known as **intermediate keys and values**. These **intermediate keys and values** are sent to **reducer**. The reduce tasks are broken into the following phases:
 - a) **Shuffle**
 - b) **Sort**
 - c) **Reducer**
 - d) **Output format**
- **Hadoop assigns map tasks** to the **datanode** where the actual data to be processed resides. This way **hadoop ensures data locality**.
(Data locality means that data is not moved over network; only computational code is moved to process data which saves network bandwidth.)

Mapper

- The **Mapper** is the first stage in a MapReduce job. It processes input data and converts it into **intermediate key-value pairs**, which will later be grouped and processed by the **Reducer**.

1) **RecordReader**:-

- Reads raw data from **HDFS** and converts it into a **structured key-value format**.
- The **key** is usually the **position** of the record, and the **value** is the **actual data**.

2) **Map Function**:-

- Processes the key-value pairs and **generates intermediate key-value pairs**.

3) **Combiner(optional)**:-

- Acts as a **mini-reducer** to optimize performance.
- It **aggregates** data locally before sending it to the reducer, **reducing network traffic**.



4) Partitioner:-

- Decides **which reducer** will process a particular key-value pair.
- Ensures that **keys with the same value go to the same reducer**.
- The partitioned data is stored **locally** before being pulled by the reducers.
Reducer 1 → (apple, 2), (banana, 1)
Reducer 2 → (orange, 1), (mango, 1)

Reducer

1) Shuffle and sort:-

- The Reducer **collects data** from all Mappers.
- It **groups the same keys together** and sorts them.

(apple, 1), (apple, 1), (banana, 1), (banana, 1), (orange, 1) **1 Mapper output**



(apple, 2), (banana, 2), (orange, 1) **1 Combiner output**

Mapper 1 Output (after Combiner): (apple, 2), (banana, 1)

Mapper 2 Output (after Combiner): (apple, 3), (banana, 2)

2 Mappers output after combiner



apple → [2, 3] → Needs final aggregation

banana → [1, 2] → Needs final aggregation

Output at shuffle phase

Shuffle → Collects intermediate outputs from all Mappers and organizes them.

Sort → Groups values by key so that the Reducer can process them efficiently.

1. The **Combiner** works only on local Mapper output, while Shuffle merges outputs from all Mappers.
2. The **Combiner** does partial aggregation, but full processing still happens in the **Reducer**.
3. The **Sort phase** ensures that the Reducer gets data in an ordered format.

2) Reduce function:-

- The **Reduce function** applies operations like **sum, average, filtering, etc.**
- It processes **one group of key-value pairs at a time.**

```
apple → [1, 1] → 2  
banana → [1, 1] → 2  
orange → [1] → 1
```

3) Output Format

- The final output is **formatted** and **saved to HDFS.**
- By default, keys and values are **separated by a tab.**

```
apple 2  
banana 2  
orange 1
```

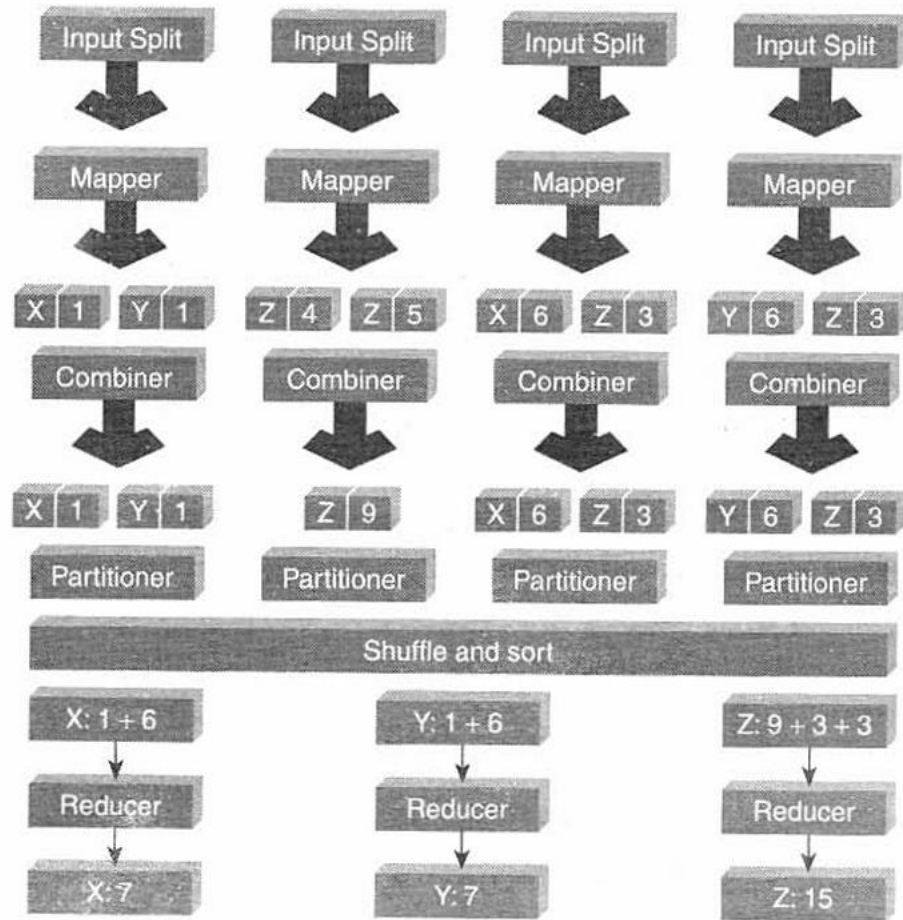


Figure 8.1 The chores of Mapper, Combiner, Partitioner, and Reducer.

Combiner

- It is an **optimization technique** for MapReduce Job. Generally, the reducer class is set to be the combiner class.
- The difference between combiner class and reducer class is as follows:
 - a) **Output** generated by the **combiner** is **intermediate data** and it is **passed to the reducer**.
 - b) **Output** of the **reducer** is **passed** to the **output file on disk**.

apple orange apple banana
orange banana apple

Input.txt

Mapper

The Mapper reads each line and splits it into words.

apple 1
orange 1
apple 1
banana 1
orange 1
banana 1
apple 1

apple 1
orange 1
apple 1
banana 1

**Mapper 1
output**

Combiner

For Mapper 1 (Line 1: apple orange apple banana)

apple 2
orange 1
banana 1

orange 1
banana 1
apple 1

**Mapper 1
output**

Combiner

For Mapper 2 (Line 2: orange banana apple)

orange 1
banana 1
apple 1

Shuffling and Sorting (before Reducer)

Input to Reducer:

Word	Values from Combiners
apple	[2, 1]
orange	[1, 1]
banana	[1, 1]



The diagram illustrates the data flow in a MapReduce job. On the left, a table lists the input to the Reducer, which consists of words and their corresponding values from combiners. An arrow points from this table to a central red rounded rectangle labeled 'Reducer'. Another arrow points from the 'Reducer' to the final output on the right, which shows the words and their summed values with the calculation for each.

Reducer

apple	3	(2 + 1)
orange	2	(1 + 1)
banana	2	(1 + 1)

Final output of
reducer

Hadoop automatically handles shuffling and sorting internally between the Combiner (or Mapper) and Reducer stages.

```

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SimpleWordCount {

    // Mapper Class
    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>
    {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens())
            {
                word.set(itr.nextToken());
                context.write(word, one); // Emit (word, 1)
            }
        }
    }
}

```

```
// Combiner Class (sums locally)
public static class SumCombiner extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        int sum = 0;
        for (IntWritable val : values)
        {
            sum += val.get(); // Local sum
        }
        context.write(key, new IntWritable(sum));
    }
}

// Reducer Class (final count)
public static class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        int total = 0;
        for (IntWritable val : values)
        {
            total += val.get(); // Final sum
        }
        context.write(key, new IntWritable(total));
    }
}
```

```

// Main Method (Driver)
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "simple word count");

    job.setJarByClass(SimpleWordCount.class);
    job.setMapperClass(TokenMapper.class);
    job.setCombinerClass(SumCombiner.class); // Optional optimization
    job.setReducerClass(SumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0])); // Input path
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // Output path

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Partitioner

- The partitioning phase happens after map phase and before reduce phase.
- Usually the **number of partitions are equal to the number of reducers.**

Purpose of using partitioner (explained by considering word count example program) :

- a) Distributes words efficiently across reducers.
- b) Improves performance by reducing workload per reducer.
- c) Reducers process alphabetically similar words together.

```

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Partitioner;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

// Mapper Class
public class WordCountWithPartitioner
{
    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>
    {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens())
            {
                word.set(itr.nextToken());
                context.write(word, one); // Emit (word, 1)
            }
        }
    }
}

```

```

// Partitioner Class (Distributes keys based on the first letter)
public static class AlphabetPartitioner extends Partitioner<Text, IntWritable>
{
    public int getPartition(Text key, IntWritable value, int numReduceTasks)
    {
        char firstLetter = Character.toLowerCase(key.toString().charAt(0));
        if (numReduceTasks == 0)
        {
            return 0;
        }
        if (firstLetter >= 'a' && firstLetter <= 'h')
        {
            return 0 % numReduceTasks; // Partition 0 for A-H
        }
        else if (firstLetter >= 'i' && firstLetter <= 'q')
        {
            return 1 % numReduceTasks; // Partition 1 for I-Q
        }
        else
        {
            return 2 % numReduceTasks; // Partition 2 for R-Z
        }
    }
}

```

Partitioning Logic

Partition 0 → Words starting with A-H

Partition 1 → Words starting with I-Q

Partition 2 → Words starting with R-Z

// Reducer Class

```
public static class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        int sum = 0;
        for (IntWritable val : values)
        {
            sum += val.get(); // Aggregate count
        }
        context.write(key, new IntWritable(sum)); // Emit final count
    }
}
```

// Driver Code

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count with partitioner");

    job.setJarByClass(WordCountWithPartitioner.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setPartitionerClass(AlphabetPartitioner.class); // Custom partitioner
    job.setReducerClass(SumReducer.class);

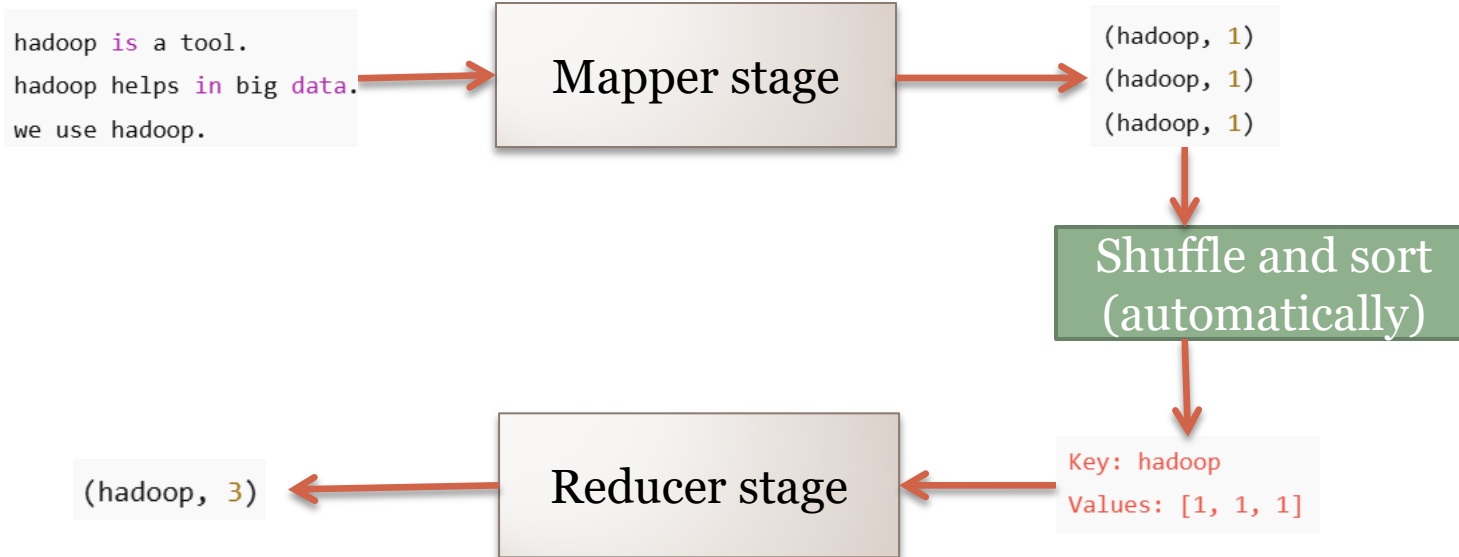
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0])); // Input path
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // Output path

    job.setNumReduceTasks(3); // Setting 3 reducers

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Searching



```

import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class KeywordSearchSimple
{
    public static class MyMapper extends Mapper<Object, Text, Text, IntWritable>
    {
        private final static IntWritable one = new IntWritable(1);
        private final static Text keyword = new Text("hadoop");
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            StringTokenizer tokenizer = new StringTokenizer(value.toString());
            while (tokenizer.hasMoreTokens())
            {
                String word = tokenizer.nextToken();
                if (word.equals("hadoop"))
                {
                    context.write(keyword, one);
                }
            }
        }
    }
}

```

```

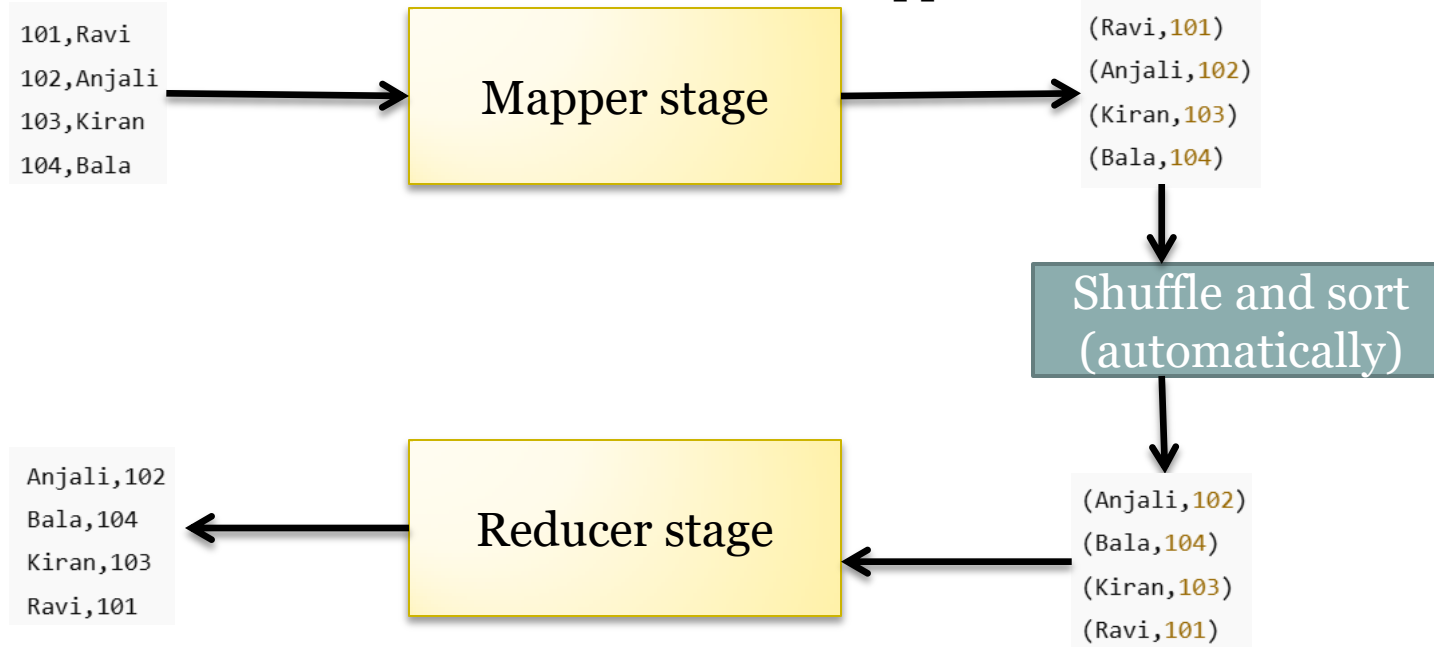
public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        int sum = 0;
        for (IntWritable val : values)
        {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception
{
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Simple Keyword Search");
    job.setJarByClass(KeywordSearchSimple.class);
    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0])); // input path
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // output path
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Sorting

- MapReduce **automatically sorts by keys**, and **student names (values) need to be sorted**, we'll **swap the key and value** in the Mapper, so names become keys.



```

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SortByStudentName
{
    public static class NameMapper extends Mapper<Object, Text, Text, NullWritable>
    {
        private Text nameKey = new Text();
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            // Assuming input format: ID,Name
            String[] parts = value.toString().split(",");
            if (parts.length == 2)
            {
                String name = parts[1].trim();
                nameKey.set(name + "," + parts[0]); // Making name as key
                context.write(nameKey, NullWritable.get());
            }
        }
    }
}

```

```

public static class NameReducer extends Reducer<Text, NullWritable, Text, NullWritable>
{
    public void reduce(Text key, Iterable<NullWritable> values, Context context) throws IOException, InterruptedException
    {
        context.write(key, NullWritable.get()); // Just emit sorted output
    }
}

public static void main(String[] args) throws Exception
{
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Sort By Student Name");

    job.setJarByClass(SortByStudentName.class);
    job.setMapperClass(NameMapper.class);
    job.setReducerClass(NameReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(NullWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0])); // Input file path
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // Output path

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Compression

In MapReduce programming, you can compress the MapReduce output file. Compression provides two benefits as follows:

1. Reduces the space to store files.
2. Speeds up data transfer across the network.

You can specify compression format in the Driver Program as shown below:

```
conf.setBoolean("mapred.output.compress", true);  
conf.setClass("mapred.output.compression.codec", GzipCodec.class, CompressionCodec.class);
```

Here, codec is the implementation of a compression and decompression algorithm. GzipCodec is the compression algorithm for gzip. This compresses the output file.